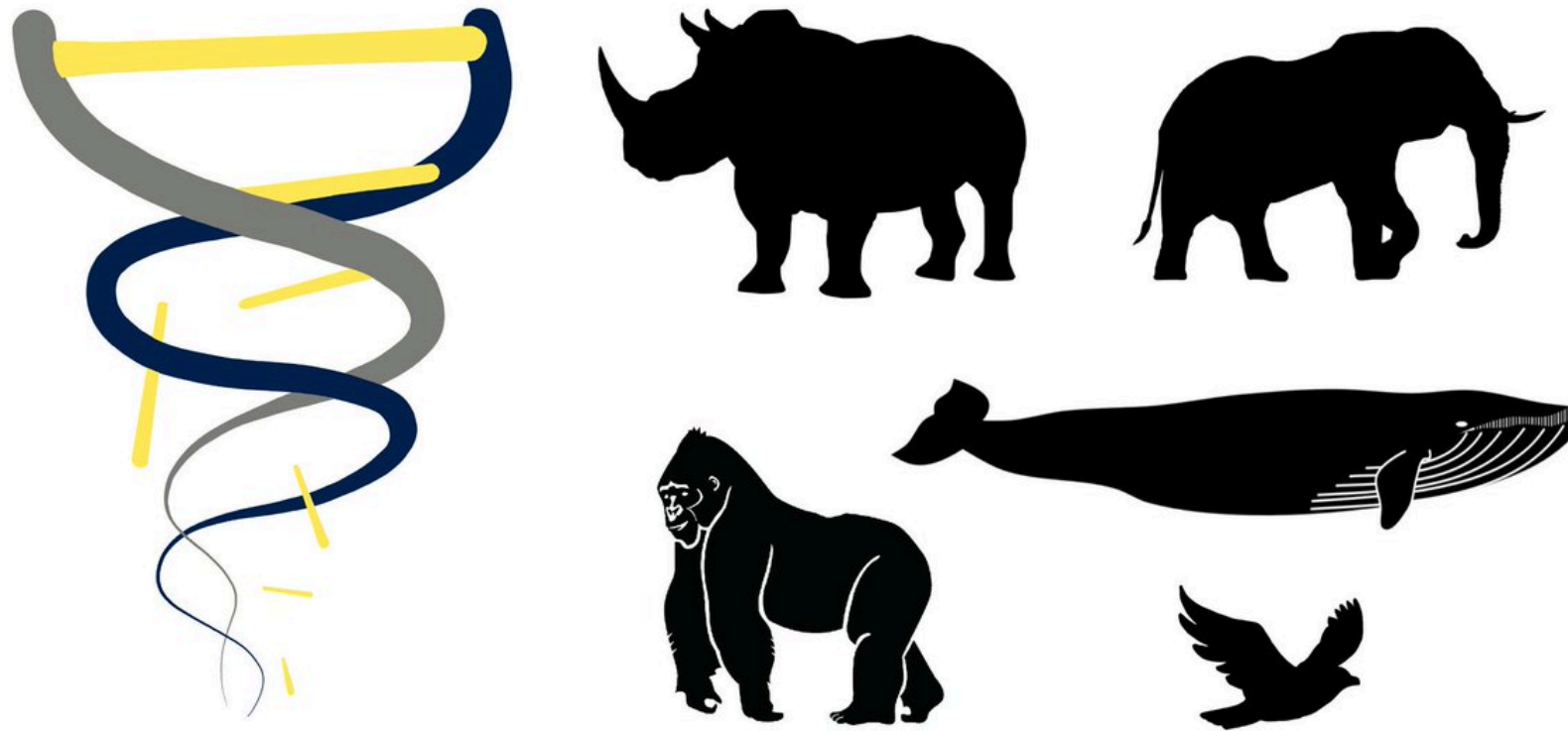


An Example Snakemake Workflow

Verena Kutschera

2026-05-25

The GenErode pipeline



<https://github.com/NBISweden/GenErode>

Kutschera et al. 2022 (BMC Bioinformatics)

The GenErode pipeline

- Developed in a NBIS project with Love Dalén's lab (Centre for Palaeogenetics, SU & NRM)
- Compares population genomics statistics from historical and modern samples of endangered populations



Skull of Sumatran rhinoceros (*Dicerorhinus sumatrensis*)
(Photo: Lee Kong Chian, Natural History Museum, Singapore,
CC BY-SA 4.0)



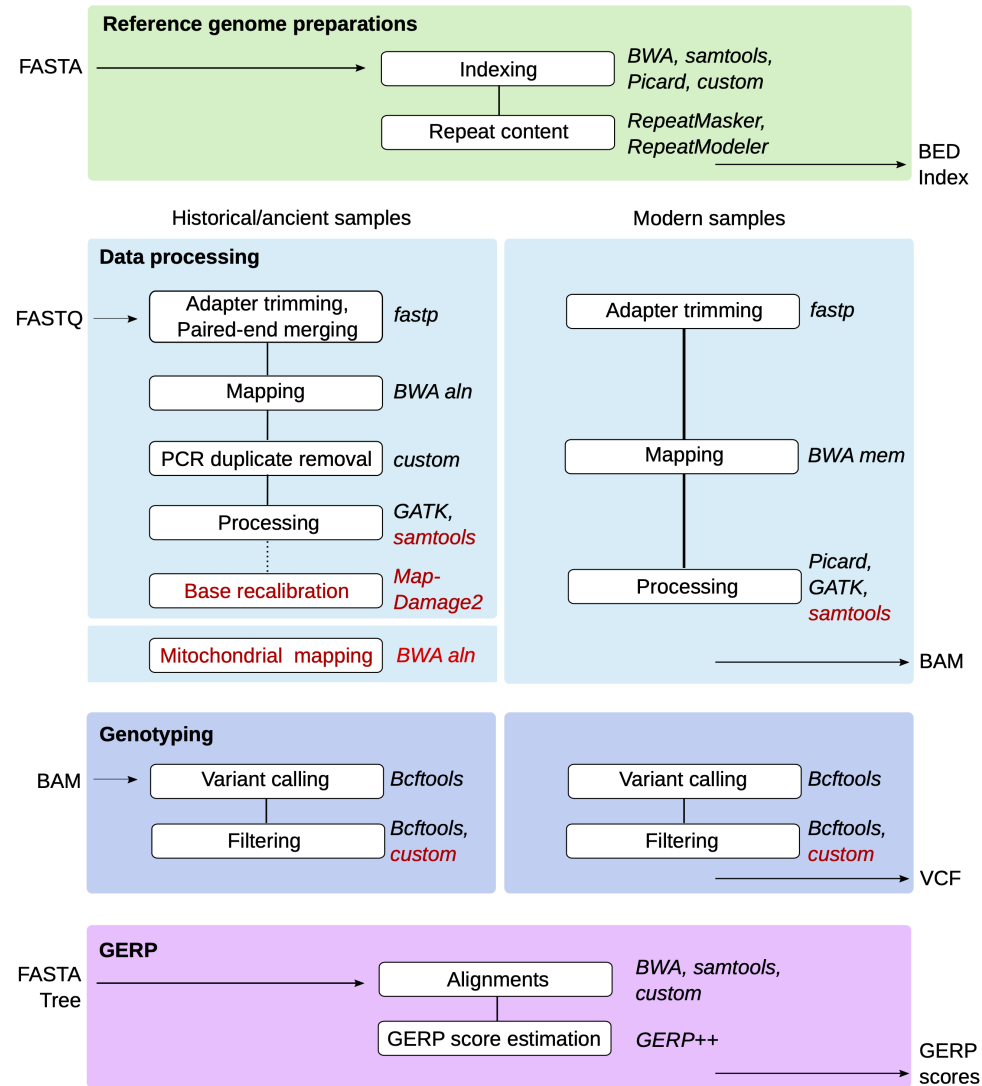
Sumatran rhinoceros mother with juvenile (Photo:
International Rhino Foundation, CC BY 2.0)

- Data processing from fastq files to BAM & VCF files plus downstream population genomics analyses

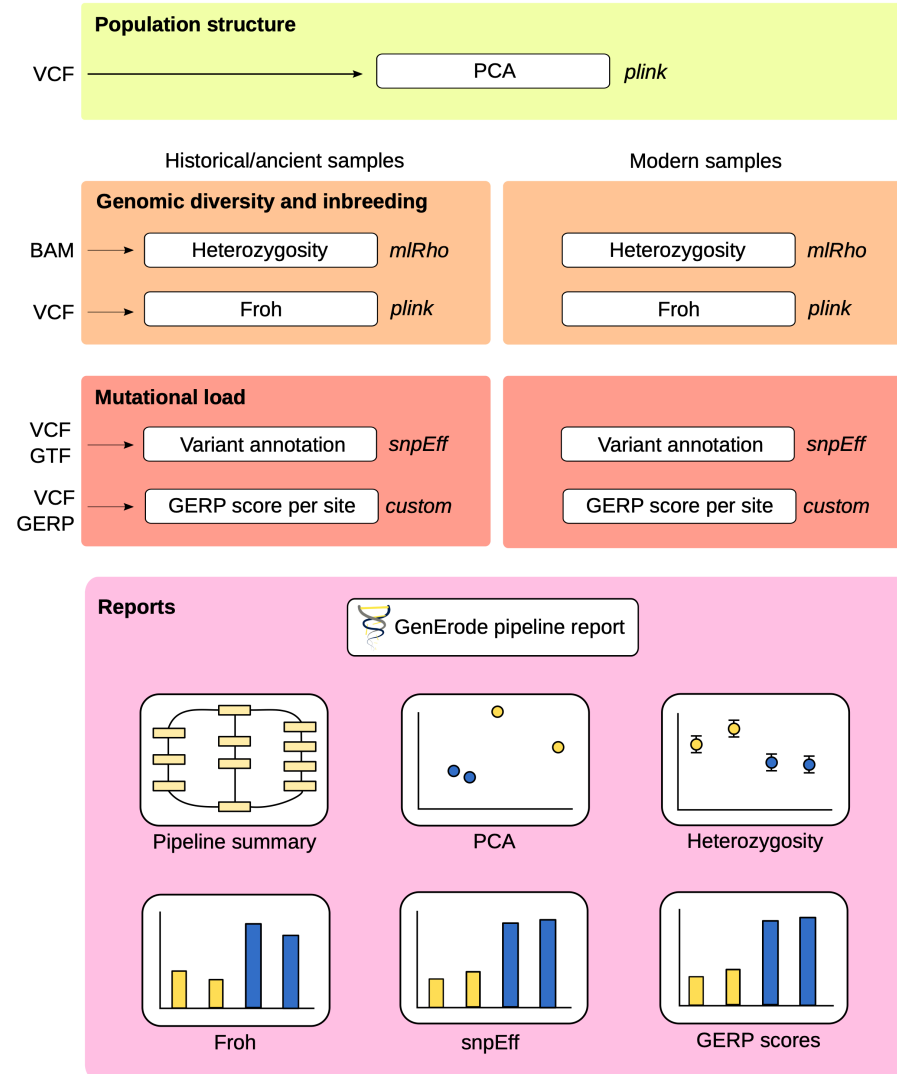
The GenErode pipeline

- Started at Snakemake version 3.10 (!), current pipeline runs with Snakemake version 8.27.1 (soon to be updated)
- Historical and modern samples are processed in parallel
- Whole-genome resequencing data from historical/ancient samples needs special processing as DNA degrades over time
- Some analyses or filtering steps are run separately for modern and historical samples, or only for historical samples

Analysis Tracks of the Workflow



Analysis Tracks of the Workflow



The Workflow Structure

- Rules with the actual analyses in separate Snakefiles (in `workflow/rules/`)
 - `workflow/rules/common.smk` contains Python code to create dictionaries & lists from metadata tables and the config file
- Snakefile
 - `include` of rule Snakefiles
 - `all` rule collecting output files produced by the different rule Snakefiles
 - Python and bash code to generate and edit the pipeline report with `snakemake --report`
- Cluster execution with slurm:
 - `config/slurm/` folder with profile YAML files for execution with the slurm plugin

The Workflow Structure

- Metadata files for historical and modern samples (separately)
 - Sample IDs, readgroup IDs, sequencing technology, paths to fastq files
- Example `historical_samples.csv` file

```
samplename_index_lane, readgroup_id, readgroup_platform, path_to_R1_f  
VK01_01_L2, BHYOX3ALTH.L2.01, illumina, data/P01_2.R  
VK01_02_L2, BHYOX3ALTH.L2.02, illumina, data/P02_2.R
```


The Workflow Structure

- Config file `config.yaml` (to be edited by users, placed in `config/`)
 - Paths to input data and metadata tables
 - Selection of analysis steps to be run
 - Parameters for different rules
 - Lists with samples for optional analyses

```
#####
# 1) Full path to reference genome assembly.
# Reference genome has to be checked for short and concise FASTA
# headers without special characters and has to be uncompressed.
# The file name will be reused by the pipeline and can have the file
# name extensions *.fasta, *.fa or *.fna.
ref_path: ""
#####
```

How to choose which steps to run

Step 1: Use booleans in the config file (`config/config.yaml`) as on/off switches

```
#####  
# FastQC on raw reads, adapter and quality trimming (incl. read merging  
# for historical samples) using fastp, FastQC on trimmed reads.  
# Adapter sequences are automatically detected.  
# Automatic detection of NovaSeq or NextSeq samples and activation of  
# poly-G tail trimming.  
fastq_processing: True  
  
[...]  
#####  
  
#####  
# Map historical and modern reads to reference genome assembly (specified  
mapping: False  
#####
```

How to choose which steps to run

For many analysis steps, parameters can be specified in the config file

```
#####  
# FastQC on raw reads, adapter and quality trimming (incl. read merging  
# for historical samples) using fastp, FastQC on trimmed reads.  
# Adapter sequences are automatically detected.  
# Automatic detection of NovaSeq or NextSeq samples and activation of  
# poly-G tail trimming.  
fastq_processing: True  
  
# Minimum read length.  
# Historical samples (after trimming and read merging)  
hist_readlength: "30" # recommended setting: 30 bp  
  
# Modern samples (after trimming)  
mod_readlength: "30"  
#####
```

How to choose which steps to run

These parameters can be used in the workflow with the syntax

`config["parameter_name"]`, e.g. `config["hist_readlength"]`

```
rule fastp_historical:
    input:
        R1="data/historical/{sample}_{index}_{lane}_R1.fastq.gz",
        R2="data/historical/{sample}_{index}_{lane}_R2.fastq.gz",
    output:
        merged="results/historical/trimming/{sample}_{index}_{lane}_trimmed.fastq.gz",
    params:
        readlength=config["hist_readlength"],
    shell:
        """
        fastp -i {input.R1} -I {input.R2} \
        -l {params.readlength} \
        --merged_out={output.merged} 2> {log}
        """
```

How to choose which steps to run

Step 2: Use some Python code, `include` and the rule `all` to figure out what the workflow will do

- The main `Snakefile` contains an empty Python list `all_outputs` to collect output files from the included rule Snakefiles,

```
all_outputs = []
```

- the `include` variable to attach the rule Snakefiles corresponding to the analysis steps that were set to `True` in the config file,

```
if config["fastq_processing"]:  
    include: "workflow/rules/1.1_fastq_processing.smk"
```

- and the rule `all` that takes the output files from the list `all_outputs` as input

```
rule all:  
    input: all_outputs,
```

How to choose which steps to run

Step 2: Use some Python code, `include` and the rule `all` to figure out what the workflow will do

- The rule Snakefile (`workflow/rules/1.1_fastq_processing.smk`) contains some Python code to add its final output files to the list `all_outputs` in the main `Snakefile`

```
import os
if os.path.exists(config["historical_samples"]):
    all_outputs.append("data/historical/stats/multiqc/multiqc_report.html")
    all_outputs.append("results/historical/trimming/stats/multiqc/multiqc_report.html")

if os.path.exists(config["modern_samples"]):
    all_outputs.append("data/modern/stats/multiqc/multiqc_report.html")
    all_outputs.append("results/modern/trimming/stats/multiqc/multiqc_report.html")
```

How to choose which steps to run

Step 2: Use some Python code, `include` and the rule `all` to figure out what the workflow will do

- `config["historical_samples"]` and `config["modern_samples"]` point to the config file where the paths to metadata files are specified

```
#####
# 2) Relative paths (from the main snakemake directory) to metadata
# files with sample information.
# Example files can be found in "config/"
historical_samples: "" # leave empty ("") if not run for historical samples
modern_samples: "" # leave empty ("") if not run for modern samples.
#####
```

How to choose which steps to run

Step 2: Use some Python code, `include` and the rule `all` to figure out what the workflow will do

- The rule Snakefile (`workflow/rules/1.1_fastq_processing.smk`) contains some Python code to add its final output files to the list `all_outputs` in the main `Snakefile`

```
import os
if os.path.exists(config["historical_samples"]):
    all_outputs.append("data/historical/stats/multiqc/multiqc_report.html")
    all_outputs.append("results/historical/trimming/stats/multiqc/multiqc_report.html")

if os.path.exists(config["modern_samples"]):
    all_outputs.append("data/modern/stats/multiqc/multiqc_report.html")
    all_outputs.append("results/modern/trimming/stats/multiqc/multiqc_report.html")
```

By using the `if` statement to check for the presence of historical or modern metadata files, the workflow can also be run only for historical or only for modern samples

Questions?

